

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

REPORT ON LABORATORY WORK NO. 5

OPERATING SYSTEMS

The File System with Your Own Hands

Done by: st. gr. FAF-241, Lucian-Adrian Gavril

Checked by: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

INTRODUCTION 3

1 THE TREE AND THE FILE TYPES 4

1.1 Five kinds of file 4

1.2 /bin is a link: the merged /usr 4

1.3 Devices are files 5

1.4 Where is the space? 5

2 INODES AND LINKS 6

2.1 A name is not the file 6

2.2 Hard links 6

2.3 Symbolic links 6

2.4 Link count of a directory 7

2.5 Size versus allocated space 8

3 PERMISSIONS 9

3.1 Typical modes 9

3.2 umask 9

3.3 r and x on a directory 10

3.4 setuid and the sticky bit 10

4 DISKS AND THE KERNEL AS FILES 11

4.1 Disks and mounts 11

4.2 /proc 11

5 STRETCH EXPERIMENTS 13

5.1 Magic numbers 13

5.2 Setuid audit 13

5.3 Deleting an open file 14

5.4 A filesystem in a file 14

CONCLUSIONS 15

REFERENCES 16

INTRODUCTION

Purpose of the work. To see from the shell how Linux organizes files: the directory tree and the file types, the difference between a *name* and an *inode*, hard and symbolic links, how space is really allocated, how permission bits are checked, and how disks and the kernel itself appear as files.

Environment. Debian GNU/Linux 13, kernel 6.12, GNU coreutils 9.7, ordinary user `dev1` (UID 1000), working directory `~/os-lab5` on an ext4 root filesystem (`/dev/sda2`). Every terminal figure is the real output of the commands shown in it.

1 THE TREE AND THE FILE TYPES

1.1 Five kinds of file

The first character of `ls -l` is the file type stored in the inode; `file` reports the same type in words (Figure 1.1).

Path	Type char	Type	Note
`/bin`	`l`	symbolic link	points to `usr/bin`; size 7 = length of “usr/bin”
`/dev/null`	`c`	character device	major 1, minor 3; no size, only device numbers
`/etc`	`d`	directory	122 links: 2 + one per subdirectory
`/etc/passwd`	`-`	regular file	2014 bytes of text, mode 644
`/tmp`	`d (t)`	directory, sticky	mode 1777

Table 1.1 – File types of the inspected paths

```
dev1@dev1:~/os-lab5$ cd ~/os-lab5
dev1@dev1:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Aug 20 21:57 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 24 08:22 /dev/null
drwxr-xr-x 122 root root 12288 Nov  2 07:22 /etc
-rw-r--r--   1 root root  2014 Oct 19 01:31 /etc/passwd
drwxrwxrwt 232 root root 28140 Nov  2 17:13 /tmp
dev1@dev1:~/os-lab5$ file /bin /dev/null /etc /etc/passwd /tmp
/bin:        symbolic link to usr/bin
/dev/null:   character special (1/3)
/etc:        directory
/etc/passwd: ASCII text
/tmp:        sticky, directory
```

Figure 1.1 – File types with ls -ld and file

1.2 /bin is a link: the merged /usr

`/bin`, `/sbin` and `/lib` are symbolic links into `/usr`, so `/bin/ls` and `/usr/bin/ls` have the same inode number, 2658541: they are one file reached by two paths (Figure 1.2). Debian merged the old “essential” `/bin` into `/usr/bin` because the split no longer matched how systems boot.

```
dev1@dev1:~/os-lab5$ ls -ld /bin /sbin /lib /usr/bin
lrwxrwxrwx 1 root root     7 Aug 20 21:57 /bin -> usr/bin
lrwxrwxrwx 1 root root     7 Aug 20 21:57 /lib -> usr/lib
lrwxrwxrwx 1 root root     8 Aug 20 21:57 /sbin -> usr/sbin
drwxr-xr-x 2 root root 53248 Oct 25 21:18 /usr/bin
dev1@dev1:~/os-lab5$ readlink -f /bin/ls ; ls -li /bin/ls /usr/bin/ls
/usr/bin/ls
2658541 -rwxr-xr-x 1 root root 158632 Jun  4  2025 /bin/ls
2658541 -rwxr-xr-x 1 root root 158632 Jun  4  2025 /usr/bin/ls
```

Figure 1.2 – The /usr merge: /bin, /sbin and /lib are links

1.3 Devices are files

In `/dev`, the size column is replaced by *major, minor*: the major number selects the driver and the minor number selects the device. `c` devices (`null` 1,3, `zero` 1,5, `random` 1,8, `tty` 5,0) are byte streams. `b` devices (the disk `sda` 8,0 and its partition `sda2` 8,2) are block-addressable. Writing to `/dev/null` discards the data; reading `/dev/zero` returns zero bytes (Figure 1.3).

```
dev1@dev1:~/os-lab5$ ls -l /dev/null /dev/zero /dev/random /dev/tty /dev/sda /dev/sda2
crw-rw-rw- 1 root root 1, 3 Oct 24 08:22 /dev/null
crw-rw-rw- 1 root root 1, 8 Oct 24 08:22 /dev/random
brw-rw---- 1 root disk 8, 0 Oct 24 08:22 /dev/sda
brw-rw---- 1 root disk 8, 2 Oct 24 08:22 /dev/sda2
crw-rw-rw- 1 root tty  5, 0 Nov  2 17:12 /dev/tty
crw-rw-rw- 1 root root 1, 5 Oct 24 08:22 /dev/zero
dev1@dev1:~/os-lab5$ echo secret > /dev/null ; head -c 8 /dev/zero | od -A d -t x1
0000000 00 00 00 00 00 00 00 00
0000008
```

Figure 1.3 – Character and block devices in /dev

1.4 Where is the space?

`/usr` uses 9.9 GiB and `/var/log` 150 MiB, about 66 times less (Figure 1.4). `/usr` holds the read-only installed software; `/var/log` holds only variable data, and it stays small because `journald` and `logrotate` cap and rotate the logs.

```
dev1@dev1:~/os-lab5$ sudo du -sh /var/log /usr 2>/dev/null
150M    /var/log
9.9G    /usr
```

Figure 1.4 – Disk usage of /var/log and /usr

2 INODES AND LINKS

2.1 A name is not the file

`stat` shows everything the inode stores: size 6, 8 blocks of 512 B, inode 7481761, 1 link, mode 0644, owner and the four timestamps (Figure 2.1). The name `notes.txt` is not among them; it exists only as an entry (name → inode) in the directory `~/os-lab5`.

```
dev1@dev1:~/os-lab5$ echo hello > notes.txt
dev1@dev1:~/os-lab5$ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 7481761     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/    dev1)   Gid: ( 1000/    dev1)
Access: 2026-11-02 17:13:35.824410262 +0200
Modify: 2026-11-02 17:13:35.824410262 +0200
Change: 2026-11-02 17:13:35.824410262 +0200
 Birth: 2026-11-02 17:13:35.824410262 +0200
dev1@dev1:~/os-lab5$ ls -i notes.txt
7481761 notes.txt
```

Figure 2.1 – The inode of notes.txt

2.2 Hard links

`ln notes.txt hard.txt` adds a second name for the same inode: both names show 7481761 and the link count rises to 2. `rm notes.txt` calls `unlink()`, which removes one name and decrements the count to 1. The data is untouched, so `cat hard.txt` still prints `hello` (Figure 2.2). The kernel frees the inode only when the link count is 0 *and* no process has the file open.

```
dev1@dev1:~/os-lab5$ ln notes.txt hard.txt
dev1@dev1:~/os-lab5$ ls -li notes.txt hard.txt
7481761 -rw-r--r-- 2 dev1 dev1 6 Nov  2 17:13 hard.txt
7481761 -rw-r--r-- 2 dev1 dev1 6 Nov  2 17:13 notes.txt
dev1@dev1:~/os-lab5$ rm notes.txt
dev1@dev1:~/os-lab5$ cat hard.txt ; ls -li hard.txt
hello
7481761 -rw-r--r-- 1 dev1 dev1 6 Nov  2 17:13 hard.txt
```

Figure 2.2 – A hard link survives the removal of the original name

2.3 Symbolic links

A symbolic link is a separate inode (7481762) whose content is a path. Its size is 8, the length of `hard.txt`. While the target exists, `cat soft.txt` follows it. After `rm hard.txt` the link dangles: `No`

such file or directory, although ls and readlink still show the stored path (Figure 2.3).

```
dev1@dev1:~/os-lab5$ ln -s hard.txt soft.txt
dev1@dev1:~/os-lab5$ ls -li hard.txt soft.txt
7481761 -rw-r--r-- 1 dev1 dev1 6 Nov  2 17:13 hard.txt
7481762 lrwxrwxrwx 1 dev1 dev1 8 Nov  2 17:13 soft.txt -> hard.txt
dev1@dev1:~/os-lab5$ cat soft.txt
hello
dev1@dev1:~/os-lab5$ rm hard.txt
dev1@dev1:~/os-lab5$ cat soft.txt
cat: soft.txt: No such file or directory
dev1@dev1:~/os-lab5$ ls -l soft.txt ; readlink soft.txt
lrwxrwxrwx 1 dev1 dev1 8 Nov  2 17:13 soft.txt -> hard.txt
hard.txt
```

Figure 2.3 – A symbolic link and how it dangles

	Hard link	**Symbolic link**
What it is	another name for the same inode	a separate inode holding a path
Inode number	same as the target	its own
Target removed	data stays, file still readable	link dangles
Across filesystems / to a directory	no / no	yes / yes

Table 2.1 – Hard link versus symbolic link

2.4 Link count of a directory

A new directory has 2 links: its name in the parent and its own . Each subdirectory adds one more through its .., so after mkdir d/x d/y the count is 4. A regular file inside does not change it (Figure 2.4). In general, links of a directory = 2 + the number of subdirectories, which is how /etc reaches 122.

```
dev1@dev1:~/os-lab5$ mkdir d ; ls -ld d
drwxr-xr-x 2 dev1 dev1 4096 Nov  2 17:13 d
dev1@dev1:~/os-lab5$ mkdir d/x d/y ; ls -ld d
drwxr-xr-x 4 dev1 dev1 4096 Nov  2 17:13 d
dev1@dev1:~/os-lab5$ touch d/file ; ls -ld d
drwxr-xr-x 4 dev1 dev1 4096 Nov  2 17:13 d
dev1@dev1:~/os-lab5$ ls -lia d | head -4
total 16
7481761 drwxr-xr-x 4 dev1 dev1 4096 Nov  2 17:13 .
7481760 drwxr-xr-x 3 dev1 dev1 4096 Nov  2 17:13 ..
7481765 -rw-r--r-- 1 dev1 dev1    0 Nov  2 17:13 file
```

Figure 2.4 – Link count of a directory

2.5 Size versus allocated space

`one.txt` contains 2 bytes (`x` and a newline) but occupies 4.0 KiB, i.e. 8 sectors of 512 B: ext4 allocates whole 4 KiB blocks. `truncate -s 1G big.img` only writes the size into the inode. `ls` reports 1.0 G, while `du` reports 0, because no block was allocated. Reading the file returns zeros generated on the fly (Figure 2.5). Such *sparse* files are how VM disk images can be “large” and still cost nothing until they are written.

```
dev1@dev1:~/os-lab5$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-r--r-- 1 dev1 dev1 2 Nov  2 17:13 one.txt
4.0K    one.txt
dev1@dev1:~/os-lab5$ stat -c '%n size=%s blocks=%b (x%B bytes)' one.txt
one.txt size=2 blocks=8 (x512 bytes)
dev1@dev1:~/os-lab5$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-r--r-- 1 dev1 dev1 1.0G Nov  2 17:13 big.img
0       big.img
dev1@dev1:~/os-lab5$ df -h . | tail -1
/dev/sda2       121G   40G   76G  35% /
```

Figure 2.5 – Logical size against allocated blocks

3 PERMISSIONS

3.1 Typical modes

Octal	Symbolic	Typical use	Observed on
600	`rw-------`	private file (keys)	`s.txt` after `chmod 600`
640	`rw-r-----`	secret readable by a group	`/etc/shadow` (group `shadow`)
644	`rw-r--r--`	public data	`/etc/passwd`
700	`rwx------`	private directory	`~/.ssh`
755	`rwxr-xr-x`	program, public directory	`/usr/bin/ls`

Table 3.1 – Common permission modes

After `chmod 000` even the owner gets `Permission denied` (Figure 3.1). The kernel picks *one* class: owner if the UIDs match, otherwise group, otherwise others. It checks only those bits, and the owner's bits are now 0. Ownership gives only the right to run `chmod`, which is how the file was made readable again. Only root bypasses the check.

```
dev1@dev1:~/os-lab5$ ls -l /etc/passwd /etc/shadow /usr/bin/ls ; ls -ld ~/.ssh
-rw-r--r-- 1 root root     2014 Oct 19 01:31 /etc/passwd
-rw-r----- 1 root shadow    978 Oct 19 01:31 /etc/shadow
-rwxr-xr-x 1 root root   158632 Jun  4  2025 /usr/bin/ls
drwx------ 2 dev1 dev1 4096 Nov  2 16:59 /home/dev1/.ssh
dev1@dev1:~/os-lab5$ echo top > s.txt ; chmod 000 s.txt ; ls -l s.txt ; cat s.txt
---------- 1 dev1 dev1 4 Nov  2 17:13 s.txt
cat: s.txt: Permission denied
dev1@dev1:~/os-lab5$ chmod 600 s.txt ; cat s.txt ; stat -c '%A %a %n' s.txt
top
-rw------- 600 s.txt
```

Figure 3.1 – Typical modes, and chmod 000 locking out the owner

3.2 umask

The umask `0022` removes write permission for group and others from every new file. Files are created from 0666 and directories from 0777: $0666 \,\&\, \neg 0022 = 0644$ and $0777 \,\&\, \neg 0022 = 0755$, exactly as `new.txt` and `newdir` show. With `umask 077` the new file is `rw-------` (Figure 3.2). Files never start with `x`, because the base mode 0666 has no execute bits.

```
dev1@dev1:~/os-lab5$ umask
0022
dev1@dev1:~/os-lab5$ touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
-rw-r--r-- 1 dev1 dev1    0 Nov  2 17:13 new.txt
drwxr-xr-x 2 dev1 dev1 4096 Nov  2 17:13 newdir
dev1@dev1:~/os-lab5$ umask 077 ; touch priv.txt ; ls -l priv.txt ; umask 022
-rw------- 1 dev1 dev1 0 Nov  2 17:13 priv.txt
```

Figure 3.2 – umask and the modes of new files

3.3 r and x on a directory

A directory's content is a list of names. `r` allows reading that list; `x` allows *using* the directory in a path to reach the inodes behind the names. With only `r`, `ls box` prints the name `f`, but `cat box/f` is denied. With only `x`, `ls` is denied, but `cat box/f` prints `hi` because the name is already known (Figure 3.3).

```
dev1@dev1:~/os-lab5$ mkdir box ; echo hi > box/f
dev1@dev1:~/os-lab5$ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied
dev1@dev1:~/os-lab5$ chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
dev1@dev1:~/os-lab5$ chmod 755 box
```

Figure 3.3 – Read and execute permission on a directory

3.4 setuid and the sticky bit

`/usr/bin/passwd` is `4755`, `rwsr-xr-x`: the `s` (setuid) makes the process run with the effective UID of the owner, root. That is why an ordinary user can change their password in `/etc/shadow`, which is `rw-r-----` and owned by root. `/tmp` is `1777`, `rwxrwxrwt`: everyone may create files, but the sticky `t` allows deleting or renaming only one's own files (Figure 3.4).

```
dev1@dev1:~/os-lab5$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 118168 Apr 19  2025 /usr/bin/passwd
drwxrwxrwt 232 root root 28140 Nov  2 17:13 /tmp
dev1@dev1:~/os-lab5$ stat -c '%a %A %n' /usr/bin/passwd /tmp
4755 -rwsr-xr-x /usr/bin/passwd
1777 drwxrwxrwt /tmp
dev1@dev1:~/os-lab5$ ls -l /etc/shadow
-rw-r----- 1 root shadow 978 Oct 19 01:31 /etc/shadow
```

Figure 3.4 – setuid on passwd and the sticky bit on /tmp

4 DISKS AND THE KERNEL AS FILES

4.1 Disks and mounts

The machine has one 1.8 TiB disk `sda` (8:0) with six partitions. The root filesystem is `sda2`, ext4, 121 GiB, 35% used, mounted `rw,relatime,errors=remount-ro`: on a disk error the kernel remounts it read-only instead of corrupting it further (Figure 4.1).

```
dev1@dev1:~/os-lab5$ lsblk -o NAME,MAJ:MIN,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/sda
NAME   MAJ:MIN   SIZE TYPE FSTYPE MOUNTPOINTS
sda      8:0     1.8T disk
├─sda1   8:1     953M part vfat   /boot/efi
├─sda2   8:2   122.9G part ext4   /
├─sda3   8:3   465.7G part ext4   /mnt/d
├─sda4   8:4      16M part
├─sda5   8:5   117.2G part ntfs
└─sda6   8:6   931.3G part ntfs   /mnt/shared
dev1@dev1:~/os-lab5$ df -h / /mnt/d
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2       121G   40G   76G  35% /
/dev/sda3       458G  288G  170G  63% /mnt/d
dev1@dev1:~/os-lab5$ findmnt -no SOURCE,FSTYPE,OPTIONS /
/dev/sda2 ext4 rw,relatime,errors=remount-ro
```

Figure 4.1 – Disks, partitions and the root mount

4.2 /proc

`/proc` is not on any disk: the kernel generates its files on every read. `/proc/cpuinfo` lists 12 logical processors (Intel Core i5-12400F, 6 cores × 2 threads), and `/proc/meminfo` reports 32 707 268 kB ≈ 31.2 GiB of RAM. `head -5 /proc/self/status` prints `Name: head`, because `/proc/self` is a link that the kernel resolves to the PID of *whoever opens it*. For `ls` it pointed to 3259196, a different process from the shell, which is 3259107 (Figure 4.2).

```
dev1@dev1:~/os-lab5$ grep -c processor /proc/cpuinfo ; grep -m1 'model name' /proc/cpuinfo
12
model name      : 12th Gen Intel(R) Core(TM) i5-12400F
dev1@dev1:~/os-lab5$ head -3 /proc/meminfo
MemTotal:       32707268 kB
MemFree:         3092372 kB
MemAvailable:   24407948 kB
dev1@dev1:~/os-lab5$ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   3259508
Ngid:   0
dev1@dev1:~/os-lab5$ ls -l /proc/self ; echo "my shell is $$"
lrwxrwxrwx 1 root root 0 Oct 24 08:22 /proc/self -> 3259509
my shell is 3259415
```

Figure 4.2 – CPU, memory and /proc/self

5 STRETCH EXPERIMENTS

5.1 Magic numbers

A gzip file starts with the bytes `1f 8b` (then `08` = deflate). `file` reads these bytes, not the name: `renamed.txt` is still recognized as gzip data (Figure 5.1). On Unix the extension is only a convention; the content determines the type.

```
dev1@dev1:~/os-lab5$ gzip -c /etc/passwd > notes.gz
dev1@dev1:~/os-lab5$ od -A x -t x1z notes.gz | head -1
000000 1f 8b 08 08 b9 48 d5 6a 00 03 70 61 73 73 77 64  >.....H.j..passwd<
dev1@dev1:~/os-lab5$ file notes.gz ; cp notes.gz renamed.txt ; file renamed.txt
notes.gz: gzip compressed data, was "passwd", last modified: Sun Oct 18 22:31:21 2026, from Unix, origin
al size modulo 2^32 2014
renamed.txt: gzip compressed data, was "passwd", last modified: Sun Oct 18 22:31:21 2026, from Unix, ori
ginal size modulo 2^32 2014
```

Figure 5.1 – The gzip magic number and file ignoring the extension

5.2 Setuid audit

13 setuid programs exist in `/usr/bin` (Figure 5.2). Each one needs root for a single, narrow task: changing account data (`passwd`, `chfn`, `chsh`, `gpasswd`, `newgrp`), switching identity (`su`, `sudo`, `pkexec`), or mounting (`mount`, `umount`, `fusermount3`, `ntfs-3g`), plus the NVIDIA module loader. Because each of them runs as root, a bug in any one is a privilege escalation, so the list should be as short as possible.

```
dev1@dev1:~/os-lab5$ find /usr/bin /usr/sbin -perm -4000 -type f | sort
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/fusermount3
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/ntfs-3g
/usr/bin/nvidia-modprobe
/usr/bin/passwd
/usr/bin/pkexec
/usr/bin/su
/usr/bin/sudo
/usr/bin/umount
dev1@dev1:~/os-lab5$ find /usr/bin /usr/sbin -perm -4000 -type f | wc -l
13
```

Figure 5.2 – Setuid programs in /usr/bin and /usr/sbin

5.3 Deleting an open file

A 50 MiB file was opened on fd 3 and then removed. `/proc/$$/fd/3` still points to it, marked `(deleted)`, and `df` shows no change in used space: the name is gone, but the inode is still referenced by the open descriptor. Only after `exec 3<&-` closes it does used space drop by $41\,239\,780 - 41\,188\,580 = 51\,200$ KiB, exactly 50 MiB (Figure 5.3). This is why deleting a large log that a running service still writes does not free disk space.

```
dev1@dev1:~/os-lab5$ head -c 50M /dev/urandom > bigfile ; df -k . | tail -1
/dev/sda2      126296388 41239816  78594876  35% /
dev1@dev1:~/os-lab5$ exec 3< bigfile ; rm bigfile
dev1@dev1:~/os-lab5$ ls -l /proc/$$/fd | grep bigfile
lr-x------ 1 dev1 dev1 64 Nov  2 17:13 3 -> /home/dev1/os-lab5/bigfile (deleted)
dev1@dev1:~/os-lab5$ df -k . | tail -1
/dev/sda2      126296388 41239816  78594876  35% /
dev1@dev1:~/os-lab5$ exec 3<&-
dev1@dev1:~/os-lab5$ df -k . | tail -1
/dev/sda2      126296388 41188616  78646076  35% /
```

Figure 5.3 – Space is freed only when the last descriptor is closed

5.4 A filesystem in a file

`mkfs.ext4` formatted a 64 MiB sparse file, and it was mounted through a loop device (`/dev/loop0`). `df -i` shows 16 384 inodes, 12 already used by the root directory, `lost+found` and reserved inodes. `dumpe2fs` explains the number: for such a small filesystem mke2fs chooses 1 KiB blocks (65 536 blocks) and one 256-byte inode per 4 KiB of space, $64\ \text{MiB}/4\ \text{KiB} = 16\,384$. The inode count is fixed at format time, so a disk can be "full" of tiny files while blocks are still free (Figure 5.4).

```
dev1@dev1:~/os-lab5$ truncate -s 64M disk.img ; mkfs.ext4 -q -F disk.img ; echo mkfs=$?
mkfs=0
dev1@dev1:~/os-lab5$ sudo mkdir -p /mnt/lab5 && sudo mount -o loop disk.img /mnt/lab5
dev1@dev1:~/os-lab5$ df -i /mnt/lab5 ; df -h /mnt/lab5
Filesystem     Inodes IUsed IFree IUse% Mounted on
/dev/loop0      16384    12 16372    1% /mnt/lab5
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop0       55M   46K   51M   1% /mnt/lab5
dev1@dev1:~/os-lab5$ sudo dumpe2fs -h disk.img 2>/dev/null | grep -E '^(Inode count|Block count|Block si
ze|Inode size|Inodes per group)'
Inode count:              16384
Block count:              65536
Block size:               1024
Inodes per group:         2048
Inode size:               256
dev1@dev1:~/os-lab5$ sudo umount /mnt/lab5
```

Figure 5.4 – An ext4 filesystem in a 64 MiB file, mounted through a loop device

CONCLUSIONS

The central idea of the lab is the separation between *names* and *inodes*. A directory is only a table of (name, inode number) pairs, and all the real information (type, size, owner, mode, times, block addresses) lives in the inode. This one fact explains all the experiments. Hard links are just more names for the same inode, so data survives `rm` until the link count and the open count both reach zero (seen both with `hard.txt` and with the deleted 50 MiB file). A directory's link count is $2+$ its subdirectories because of `.` and `..`. A symbolic link is a small separate file with a path and can dangle.

The size of a file and the space it uses are independent quantities: 2 bytes cost 4 KiB, and a 1 GiB sparse file costs nothing. Permissions are checked against exactly one class, which is why `chmod 000` locks out even the owner. On a directory, `r` and `x` are two different rights (listing the names and passing through them). setuid and the sticky bit are the controlled exceptions that make passwords and `/tmp` work safely.

Finally, "everything is a file" proved literal: disks are block devices with major and minor numbers, the kernel publishes its state through `/proc`, and a whole ext4 filesystem fit inside a regular file and was mounted like a disk.

REFERENCES

1. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press. Ch. 14–15, 18.

2. The Linux Kernel documentation: *ext4 Data Structures and Algorithms*; `man 5 proc`; `man 8 mke2fs`.

3. Linux Foundation. *Filesystem Hierarchy Standard* 3.0.